

iSAQB Module: Formal Methods - VeriFast

Active Group GmbH

Created: 2026-10-04 Sun 12:48

VeriFast

- semi-automated modular formal program verification tool
- verification of single-threaded and multi-threaded C, Rust and Java programs
- uses
 - separation logic
 - SMT solvers: Z3 Theorem Prover, Redux
- written in OCaml
- Industrial case studies: <https://doi.org/10.1016/j.scico.2013.01.006>
- Repo: <https://github.com/verifast/verifast>
- DOI: <https://doi.org/10.5281/zenodo.4705415>

Running VeriFast

operating systems:

- Windows
- Linux
- macOS

user interfaces:

- VeriFast IDE
- VeriFast Visual Studio Code Extension
- VeriFast command-line tool

C Program Verification

VeriFast checks for:

- illegal memory accesses
- data races (certain type of concurrency errors)
- compliance of pre- and postconditions – provided by the programmer

Memory Representation

- based on Separation Logic
- Separation Logic is based on Hoare Logic

Hoare Logic

- Hoare triple: $\{P\}S\{Q\}$
- no precondition: $\{true\}S\{Q\}$
- S does not terminate (one option): $\{P\}S\{false\}$

Separation Logic - Proof Form

$\{precondition\}code\{postcondition\}$

Examples:

$\{emp\}x = alloc()\{x \mapsto _ \}$

$\{x \mapsto _ \}x = 5\{x \mapsto 5\}$

$\{x \mapsto _ \}free(x)\{emp\}$

Separation Logic - Logical Operators

- separation conjunction: $*$ (speak: and separately)

$$P * P \neq P \text{ vs. } P \wedge P = P$$

$x \mapsto v * x \mapsto v$ is always false

- separating implication: $\multimap$ (also called: magic wand)

$$(x \mapsto _) * ((x \mapsto 3) \multimap Q)$$

Error Detection with VeriFast

- VeriFast uses a symbolical execution of functions/methods
 - executing a function call uses only its pre- and postconditions, not its body
 - each loop is symbolically executed only once, applying the loop invariant
- VeriFast maintains a symbolic state consisting of:
 - a symbolic heap, containing permissions for accessing certain memory locations,
 - a symbolic store, assigning a symbolic value to each local variable, and
 - a path condition, a set of assumptions about the values of the symbols used in the symbolic state

Resources VeriFast

- Repository: <https://github.com/verifast/verifast>
- DOI: <https://doi.org/10.5281/zenodo.4705415>
- Introduction: <https://github.com/verifast/verifast/blob/master/intro-c.md>
- Tutorial: <https://doi.org/10.5281/zenodo.887906>
- Papers: <https://people.cs.kuleuven.be/~bart.jacobs/verifast/>
- Documentation (work in progress): <https://verifast.github.io/verifast-docs/>
- Frédéric Vogels' phd-thesis: <https://lirias.kuleuven.be/retrieve/204848>
- Featherweight VeriFast: [https://doi.org/10.2168/LMCS-11\(3:19\)2015](https://doi.org/10.2168/LMCS-11(3:19)2015)
- Separation logic in VeriFast: <http://dx.doi.org/10.5381/jot.2026.25.1.a4>

Resources Hoare logic and Separation Logic

- Hoare, C.A.R. (1969). An axiomatic basis for computer programming.
<https://doi.org/10.1145/363235.363259>
- Reynolds, J.C. (2008). An Introduction to Separation Logic (Preliminary Draft).
<https://flint.cs.yale.edu/cs428/doc/seplogic08.pdf>
- Reynolds, J.C. (2008). An Overview of Separation Logic.
https://doi.org/10.1007/978-3-540-69149-5_49
- Reynolds, J.C. (2009). An Introduction to Separation Logic
<https://doi.org/10.3233/978-1-58603-976-9-285> (no open access)
- O'Hearn, P.W. (2012). A Primer on Separation Logic (and Automatic Program Verification and Analysis).
<http://www0.cs.ucl.ac.uk/staff/p.ohearn/papers/Marktoberdorf11LectureNotes.pdf>
- Apt, K.R. & Olderog E.-R. (2019). Fifty years of Hoare's logic.
<https://doi.org/10.1007/s00165-019-00501-3>
- O'Hearn, P. (2019). Separation logic. <http://dx.doi.org/10.1145/3211968>
- Charguéraud, A. (2020). Separation Logic for Sequential Programs (Functional Pearl). <https://doi.org/10.1145/3408998>
- Wikipedia: <https://de.wikipedia.org/wiki/Hoare-Kalk%C3%BCl> /
https://en.wikipedia.org/wiki/Hoare_logic